

Fynla — Agents Map

Stamp: commit `a7c13608a`, branch `fix/rsu-value-labels-free-tier-capture`, 2026-09-22. **Method:** every claim cites a file and line read in this run (by me or by a read-only Explore agent whose report I checked against the files). Anything not read says "I COULD NOT VERIFY". No code was changed. Adjacent defects found while mapping are listed in §6 and were not fixed.

0. In plain English

Three different things in Fynla are called "agents", and they sit at three different heights:

- 1. **Module agents** are PHP classes inside the product. Each owns one planning area (protection, savings, investment, retirement, estate, goals, tax) and a Coordinating agent sits over them for the household view. They have no personality. They do the arithmetic, generate recommendations and build what-if scenarios, and both the dashboard and Fyn read their output.
- 2. **Fyn** is the AI companion. It is one assistant with one personality and one endpoint, but it runs in two write states: an onboarding state that can enter data, and an advice state that can only read. Fyn does not calculate; it explains what the module agents calculated, and it remembers through a four-store memory that holds pointers to live data rather than copies.
- 3. **Workforce agents** are the Claude Code agent definitions that build, test, review and govern Fynla itself. They have named roles, a written constitution, a board, an event log and hand work to each other under rules about who may verify what.

1. Module agents (`app/Agents/`)

Nine classes, 11,969 lines (`wc -l`). One abstract base, seven module agents, one coordinator. These have no personality: they are calculation engines with a shared contract, a shared cache convention and a shared recommendation shape. Every one is called by the web dashboard, the `/m` dashboard and Fyn.

1.1 The shared contract: `BaseAgent` (155 lines)

Member	Line	What it does
<code>analyze(int \$userId): array</code>	<code>BaseAgent.php:26</code>	Abstract. Analyse the user's data and return insights.
<code>generateRecommendations(array \$analysisData): array</code>	<code>:31</code>	Abstract. Turn an analysis into recommendations.
<code>buildScenarios(int \$userId, array \$parameters): array</code>	<code>:36</code>	Abstract. What-if scenarios.
<code>remember, rememberForUser, getUserCacheKey</code>	<code>:45, :128, :58</code>	<code>Cache::remember</code> with a versioned per-user key <code>v1_{agentclass}_{userId}_{suffix}</code> and a default TTL of <code>TaxDefaults::CACHE_TTL_STANDARD (:21)</code>
<code>invalidateUserCache, invalidateCacheForUsers, clearUserCache</code>	<code>:86, :111, :71</code>	Standardised invalidation (joint accounts loop both users)
<code>response(), roundToPenny()</code>	<code>:138, :151</code>	The envelope <code>['success', 'message', 'data', 'timestamp']</code> and penny rounding

Shared by inheritance: `FormatsCurrency (:13)`. All eight concrete agents extend it (`GoalsAgent.php:18`, `RetirementAgent.php:39`, `EstateAgent.php:32`, `SavingsAgent.php:31`, `ProtectionAgent.php:21`, `TaxOptimisationAgent.php:18`, `InvestmentAgent.php:31`, `CoordinatingAgent.php:125`).

1.2 The seven module agents, one by one

Agent	Lines	What it does (from the body)	Services it calls	Who calls it
ProtectionAgent	457	Opens with a data-readiness gate that returns a fully-keyed null response when the facts are missing (<code>:46–64</code>); then gap analysis, adequacy, coverage and a <code>missing_for_quality_advice</code> list so Fyn asks rather than guesses. <code>generateRecommendations (:348)</code> re-reads what <code>analyze</code> already built.	<code>CoverageGapAnalyzer</code> , <code>AdequacyScorer</code> , <code>RecommendationEngine</code> , <code>ScenarioBuilder</code> , <code>ProfileCompletenessChecker</code> , <code>RecommendationPersonaliser</code> , <code>ProtectionDataReadinessService</code> , <code>CrossModuleAssetAggregator</code> , <code>LifeCoverReach (:26–38)</code>	<code>CoordinatingAgent.php:148</code> , <code>ProtectionController.php:52</code> , <code>ModuleSummaryController.php</code> , <code>RecommendationsAggregatorSe</code> , <code>ProtectionPlanService.php:1</code> , <code>WhatIfScenarioService.php:4</code> , <code>DashboardAggregator.php:18</code> , <code>MobileDashboardAggregator.p</code>
SavingsAgent	607	Cash total, emergency-fund runway against resolved expenditure, ISA allowance use, liquidity, Financial Services Compensation Scheme exposure, rate comparison. The readiness branch still returns cash total and account count because the dashboard	<code>EmergencyFundCalculator</code> , <code>ISATracker</code> , <code>GoalProgressCalculator</code> , <code>LiquidityAnalyzer</code> , <code>RateComparator</code> , <code>SavingsDataReadinessService</code> , <code>SavingsStore</code> , <code>CrossModuleAssetAggregator</code> , optional <code>PSACalculator</code> , <code>FSCSAssessor</code> ,	<code>CoordinatingAgent.php:150</code> , <code>SavingsController.php:54</code> , <code>ModuleSummaryController.php</code> , <code>RecommendationsAggregatorSe</code> , <code>SavingsStrategySource.php:2</code> , <code>SavingsPlanService.php:17</code> , <code>InvestmentPlanService.php:5</code>

Agent	Lines	What it does (from the body)	Services it calls	Who calls it
		card reads them and a new spouse was seeing zero (:69–82). Joint accounts come through SavingsStore and are fractioned (:92–95). Recommendations are database-driven via SavingsActionDefinitionService with goals sorted first (:296).	PlanConfigService , GoalProgressService (:38–52); TaxConfigService at runtime (:547)	WhatIfScenarioService.php:4 dashboard aggregators
InvestmentAgent	433	Portfolio value, returns, allocation, diversification, risk, fees against a low-cost comparison, tax efficiency, deviation from target. Its distinguishing logic is holdingsAtUserShare (:44–63): holdings carry no ownership columns, so each is scaled by the owner's fraction or a joint portfolio would reach every figure whole. getPortfolioProjections (:418) passes through to InvestmentProjectionService .	PortfolioAnalyzer , DiversificationAnalyzer , MonteCarloSimulator , SimpleAssetAllocationOptimizer , FeeAnalyzer , TaxEfficiencyCalculator , TaxConfigService , InvestmentActionDefinitionService , DataReadinessService , CrossModuleAssetAggregator (:35–46)	CoordinatingAgent.php:149 , InvestmentController.php:71 on every write, for both joint owner: InvestmentProjectionControl ModuleSummaryController.php RecommendationsAggregatorSe InvestmentStrategySource.ph services, both dashboard aggregat
RetirementAgent	838	Projects defined contribution, defined benefit and State Pension income to and through retirement; Annual Allowance check; contribution optimisation; decumulation. The closed readiness gate still returns completeness actions and a provision summary so the card can say what the household holds (:87–99). Borrows the Investment module's portfolio services to analyse the pension portfolio (analyzeDCPensionPortfolio , :827). Four scenarios plus comparison (:594–709).	PensionProjector , AnnualAllowanceChecker , PensionContributionOptimizer , DecumulationPlanner , PensionPortfolioAnalyzer , TaxConfigService , RetirementActionDefinitionService , RiskPreferenceService , RetirementDataReadinessService , StatePensionAgeResolver , PortfolioAnalyzer , MonteCarloSimulator , SimpleAssetAllocationOptimizer , FeeAnalyzer , TaxEfficiencyCalculator , FutureValueCalculator (:45–69)	CoordinatingAgent.php:151 , RetirementController.php:60 ModuleSummaryController.php RecommendationsAggregatorSe RetirementStrategySource.ph RetirementPlanService.php:2 WhatIfScenarioService.php:4 dashboard aggregators; MobileDashboardAggregator.p every /m retirement figure comes fr alone
EstateAgent	1,856	The largest. Loads the estate (assets, properties, liabilities, mortgages, spouse, family, trusts, gifts), gates on readiness, then caches the Inheritance Tax picture, will review, life cover, pension amendment and goal liquidity. generateRecommendations (:489) runs a strictly ordered seven-step mitigation strategy, trusts last (:480–487): charitable-bequest rate reduction, liquidity, existing life cover, annual gifting, life cover strategy, potentially exempt transfers, then trust. Five scenarios (:1723–1819).	IHTCalculationService , EstateAssetAggregatorService , ComprehensiveEstatePlanService , GiftingStrategyOptimizer , PersonalizedTrustStrategyService , WillAnalysisService , TaxConfigService , RecommendationPersonaliser , EstateDataReadinessService , LifeCoverCalculator , LifeCoverReach , FutureValueCalculator (:44–58)	CoordinatingAgent.php:152 , ModuleSummaryController.php EstatePlanService.php:21 , WhatIfScenarioService.php:4 dashboard aggregators. Absent fr RecommendationsAggregatorSe the other five.
GoalsAgent	542	The only module agent with no readiness service (PrerequisiteGateService.php:166 : "GoalsAgent checks has_goals directly"). Reads goals for the user or joint owner, splits active from completed, groups by module, computes progress, affordability and risk. Recommendations are hand-written rules (:151): getting-started, behind-schedule, affordability, emergency-fund. getDashboardOverview (:340) gives the top goals by priority.	GoalAssignmentService , GoalAffordabilityService , GoalProgressService , GoalRiskService (:22–27); no TaxConfigService	CoordinatingAgent.php:153 , GoalsController.php:40,104 , ModuleSummaryController.php RecommendationsAggregatorSe GoalPlanService.php:19 , MobileDashboardAggregator.p DashboardAggregator or WhatIfScenarioService .
TaxOptimisationAgent	130	The thinnest. Asks TaxOptimisationService for allowance usage and strategies and republishes them with the active tax year from TaxConfigService (:11–17).	TaxOptimisationService , TaxConfigService (:20–23)	CoordinatingAgent.php:154,2 ModuleSummaryController.php TaxOptimisationController.p (routes api.php:1153–1154)

Agent	Lines	What it does (from the body)	Services it calls	Who calls it
		Three scenarios: maximise ISA, maximise pension, staged capital gains (:80). Deliberately kept off the dashboard path because it would add a cost to every dashboard load (DailyInsightService.php:41).		

1.3 CoordinatingAgent (6,951 lines): two jobs in one class

The docblock says so itself (CoordinatingAgent.php:118–124). It is both the cross-module orchestrator and, through use HasAiChat and use HasAiGuardrails (:127–128), Fyn's single tool-execution entry point.

Job one: orchestration. orchestrateAnalysis (:365–443) is the spine: emit an EngineCalled eval event, collect all seven module analyses in fixed order (collectModuleAnalysis, :645–682, each wrapped in safeModuleAnalysis at :687–696 so one failing module substitutes a typed default rather than failing the plan), ask CashFlowCoordinator for the available surplus, extract recommendations, identify and resolve conflicts via ConflictResolver (exactly three types: protection versus savings, cash flow, ISA allowance, :477–514), rank via PriorityRanker (:521–524), extract monthly demands per category and optimise the allocation, identify shortfalls, generate cross-module strategies via CrossModuleStrategyService (:407). generateHolisticPlan (:451–470) layers HolisticPlanner and an action plan on top. Net worth is not computed here; it is injected as NetWorthService (:157).

Because the agents disagree on envelope (Protection, Retirement, Estate and TaxOptimisation return response(); Savings, Investment and Goals return raw arrays), mappedModuleAnalysis (:534) is "the one place a module's raw agent output becomes the shape the prompt builder reads" (:526–533), with five mappers at :793–873. The same unwrap logic recurs in DashboardAggregator.php:126–224, MobileDashboardAggregator.php:161 and ToolResultContract.php:115.

One ranking: seeded priority wins. rankRecommendations delegates entirely to app/Services/Coordination/PriorityRanker.php, whose docblock (:8–18) states the rule: the label an action definition was seeded with sets the band (BANDS: critical 95, high 85, medium 60, low 45, :23); numeric benefit and module weight (MODULE_WEIGHTS, max 80, :26–34) contribute at most 0.8 and "can never lift a rec across one" band. Two consumers read it: this agent (Fyn's financial context, get_recommendations, the holistic plan) and RecommendationsAggregatorService.php:226 (web API, dashboard focus cards, /m next actions).

Job two: Fyn's tools. executeTool (:932–1231) normalises xAI quirks (the string "null", HTML entities, a zero sentinel for joint_owner_id, :949–969), checks the eval bypass gate and preview flag (:980–981), appends an audit-chain row (:986–993), emits an AgentDecision eval event (:996–1007), then dispatches through a match of roughly 50 tool names (:1166–1231). handleRecommendations (:2737–2772) is where ranking meets Fyn.

Constructor. Twenty-three injected services (:142–169): five coordination services, all seven module agents (:148–154), then TaxConfigService, AiToolDefinitions, NetWorthService, PrerequisiteGateService, ComposedTaxPlanService, TeaserGate, CaptureAccuracyGate, HouseholdProvisioner, SubscriptionStatusService, HouseholdExpenditureWriter, DependantsReach; roughly 100 further classes imported for runtime use (:7–116).

Cache. It does not cache its own orchestration. invalidateUserCache (:186–190) also calls CacheInvalidationService::invalidateForUser, described at :171–185 as "the belt to that observer's braces" since UserDataCacheObserver now invalidates on every financial model write (W-0239).

1.4 How the output reaches each surface

Consumer	Path	Evidence
Fyn	Three paths, all through CoordinatingAgent: FynLoop.php:92 injects it, sets per-turn state and streams on the same instance (:133–176); OnboardingChatDirector.php:115 injects it and calls executeTool at seven sites; HasAiChat.php:1622,1665 pass analyzeRelevantModules as the analysis closure. analyzeRelevantModules (:220–289) sizes the work: holistic runs the full orchestration, module fans out only to the classified modules then ranks, factual runs nothing, because the previous behaviour burned holistic compute on every chat send (:200–219).	as cited
Web dashboard	DashboardAggregator.php:17–22 injects the six module agents directly (not the coordinator) and calls analyze() on each inside its own try/catch (:118–243).	app/Services/Dashboard/DashboardAggregator.php
/m dashboard and module pages	MobileDashboardAggregator.php:55–60 does the same; ModuleSummaryController.php:96–102 serves the /m module pages, cached as mobile_module_{module}_{userId} for 86,400 seconds (:63–65) and stripping financial-quality scores (:73, Rule 12).	as cited
Holistic plan	The only HTTP surface reaching the coordinator's cross-module output: HolisticPlanningController.php:46,73 calls orchestrateAnalysis and generateHolisticPlan behind the holistic.full middleware (routes/api.php:1098–1113). DailyInsightService.php:61 also calls analyze.	as cited

Consumer	Path	Evidence
Recommendations API, focus cards, /m next actions	<code>RecommendationsAggregatorService</code> injects six module agents (:33–38, not <code>TaxOptimisation</code> , not <code>Estate</code>) and ranks through <code>PriorityRanker</code> (:226).	as cited

1.5 What the module agents share

- **The contract:** `analyze`, `generateRecommendations`, `buildScenarios` on every agent.
- **TaxConfigService:** injected by `Estate`, `Investment`, `Retirement`, `TaxOptimisation` and `Coordinating`; resolved at runtime by `Savings`; not used by `Protection` or `Goals`.
- **CalculatesOwnershipShare:** `Investment` (:33) and `Savings` (:33); `Estate` reaches joint records through `EstateAssetAggregatorService`.
- **The readiness gate:** five agents open `analyze()` with a `DataReadinessService::assess()` and return a fully-keyed null response when `can_proceed` is false (`Protection` :46–64, `Savings` :66–89, `Investment` :87–107, `Retirement` :79–104, `Estate` :84–102). `Goals` and `TaxOptimisation` have none by design (`PrerequisiteGateService.php`:166,183).
- **missing_for_quality_advice:** all six module agents (not `TaxOptimisation`) compute and publish it so `Fyn` can ask for what is absent.
- **Eval telemetry:** every agent emits `EngineCalled` around `analyze` and `generateRecommendations`; `Protection` and `Retirement` also emit `GateChecked`; `Coordinating` emits `AgentDecision`.
- **The recommendation shape** is an array convention, not a class: a `priority` or `impact` label, `title`, `description`, `action`, `category` or `type`, and one benefit key read in order `estimated_impact`, `estimated_saving`, `estimated_annual_tax_saved`, `potential_benefit` (`PriorityRanker.php`:40); the ranker adds `module`, `priority_score`, `urgency_score`, `impact_score`, `impact_label`, `timeline` (:66–74).
- **Recommendation generation splits three ways:** database-driven action definitions (`Savings`, `Investment`, `Retirement`), pass-through of the analysis (`Protection`), hand-written rules (`Goals`, `Estate`'s seven steps, `TaxOptimisation`'s mapping).

2. Fyn, the AI agent

2.1 What Fyn is

One assistant, one system prompt, one endpoint, two write states. The user never sees the switch. The canonical contract is `.claude/skills/fyn-architecture/SKILL.md` and every clause of it was re-confirmed in code this run.

Piece	Class and entry point	Writes?	Model
Advice Fyn (read-only)	<code>app/Services/AI/AdviceFyn.php:217 handle()</code>	No	Active provider's chat model
Onboarding / campaign Fyn (the only writer)	<code>app/Services/Onboarding/OnboardingChatDirector.php:188 handleUserMessage()</code> , :7876 <code>handleInlineCapture()</code> , :697 <code>handleAction()</code> , :166 <code>emitFirstTurn()</code>	Yes	Active provider, or no model at all on deterministic turns
Turn planner	<code>app/Services/AI/Loop/Planner.php:73 plan()</code>	No	Its own call with a forced <code>plan</code> tool (:208–213)
Shared turn loop	<code>app/Services/AI/Loop/FynLoop.php:194 run()</code> , :113 <code>stream()</code> , :618 <code>interceptHandoff()</code>	Routes both	n/a
Query classifier	<code>app/Services/AI/QueryClassifier.php:65</code>	No	Deterministic regex, no model
Write-intent classifier	<code>app/Services/AI/WriteIntentClassifier.php:192</code>	No	Deterministic regex, no model
Conversation summariser	<code>app/Services/AI/ConversationSummariser.php:57</code>	Index columns only	Direct HTTP to xAI (:45,137)
Proposed-fact synthesiser	<code>app/Services/AI/Learning/ProposedFactSynthesiser.php:21</code>	Stages a pending proposal	Direct HTTP to xAI (:18,42)
Advice review service	<code>app/Services/AI/AdviceReviewService.php:23</code>	No	No model; pure database comparison

The two states are distinguished by `SessionMode` (`app/Services/AI/Loop/SessionMode.php:23–39`): advice maps to persona `advice`, onboarding to `data_capture`. Inside the write state there are further per-turn modes: asset capture (`FynCaptureTurnInstructions.php:24`), verify/edit (`FynVerifyEditTurnInstructions.php:38`), inline-capture handoff (`OnboardingChatDirector.php:92`, 7968–7971) and grouped extraction (`AiToolDefinitions.php:409`).

2.2 Personality

The voice lives in one static string, `app/Services/AI/Fyn/FynSystemPrompt.php:70–216`, with only two placeholder substitutions so it is byte-identical per user and prefix-cacheable.

- **Identity** (:76–80): "You are Fyn, a UK personal-finance guidance tool inside the Fynla app... You do NOT give personalised regulated financial advice." Fyn is framed as a tool, not a professional; the earlier "thinks like a planner" framing was removed because it implied a regulatory status Fynla does not hold

(Prompts/CoreIdentity.php:12–15 docblock).

- **Personality** (PERSONALITY, :48–58): "Warm, encouraging, and clear — like a knowledgeable friend who understands financial planning deeply"; "Celebrate progress"; "Be honest about gaps or risks without being alarming"; "Never be condescending or make the user feel bad about their financial position"; "British spelling. Currency in £. Calm, plain-English tone — never patronising, never alarmist"; "Always signpost regulated advice when the user's query asks 'what should I do?'".
- **Response shape** (RESPONSE_FORMAT, :60–68): concise, bold for key figures, numbered lists for sequences, always end on a natural follow-up question, never open with "Certainly!", "Of course!", "Great question!" or "Absolutely!", first name occasionally but "do not overdo it".
- **Security** (:83–93): nine non-negotiable rules including the single canned refusal string (FynSystemPrompt::CANNED_REFUSAL, :26), with two carve-outs: a legitimate request to manage the user's own data is never an attack, and neither is a message that answers a question Fyn asked.
- **Scope** (:95–99): personal finance only; anything else is politely redirected.
- **Language rules** (:110–122): British English; no acronyms except ISA; currency as £1,250.00; never show record IDs, route paths, [Context: blocks or tool metadata; never use "waterfall", "opportunity cost", "phased approach" or similar jargon; never mention a concept that does not apply to this user.
- **Regulatory compliance** (:126–132): mandatory hedging ("you may want to consider", never "you should"); no product, provider, fund or platform names; signpost a regulated adviser; investment risk warnings; tax caveats; no market timing; never state a tax figure from memory, always the `get_tax_information` tool.
- **Read-only declaration and handoff guidance** (:149, :198–204): in advice mode every add, change or delete intent is routed through `delegate_to_capture` rather than refused.
- **FCA six-step process** (:223–242), spliced in through `{{FCA_PROCESS}}`; the writable-record vocabulary is one constant (WRITABLE_RECORD_TYPES, :41).

Two more voice rules live in the procedural corpus and are injected as overlays (FynContextAssembler.php:204–214): "Answer the user first. A user question is never acknowledged-and-advanced past" (fyn-memory/procedural/system_prompt_overlay/general/a1-answer-first.md:10–16) and "No standalone acknowledgement bubbles... only when a write actually occurred" (a2-ack-hygiene.md:10–16). Per-turn voicing rules in the assembler (FynContextAssembler.php:634–645) state mechanical claims directly with working shown, hedge judgement claims, surface at most one extra strategy after the answer, and ask one clarifying question before computing from an ambiguous figure.

The constitution's voice file names this the "Companion" register: "Warm, conversational, calm. Ends on a question. No filler openers. Closed — fully specified in CoreIdentity.php" (04-voice.md §3). The live home is FynSystemPrompt.php; CoreIdentity.php is the legacy copy (see §6).

User free text is sanitised at every interpolation site by Prompts/UserContentSanitiser.php:68–78, a denylist plus `<user_provided>` wrapping so non-ASCII names survive.

2.3 Providers

xAI is live in this environment (`.env AI_PROVIDER=xai`); the config default is Anthropic (config/services.php:54).

- **xAI**: `app/Services/AI/XaiClient.php:24–121` wraps the OpenAI PHP SDK against `https://api.x.ai/v1` (config/services.php:49), 120-second timeout, `x-grok-conv-id` header for prompt-cache routing (:42–59, 77). Model `grok-4.3` for chat, advanced and vision (config/services.php:42–48). `degrade_chat_model` is unset, so soft-degrade keeps the standard model and chat stays open.
- **Anthropic**: resolved from the container as `AnthropicClient` (Planner.php:106); `claude-haiku-4-5-20251001` chat, `claude-sonnet-4-6-20260320` advanced (config/services.php:36–37).
- **Choice**: one canonical reader `HasAiGuardrails.php:64–79` (versioned cache key, legacy key, then config); the admin toggle writes all three (AdminController.php:708–712); the chat loop captures the provider once per turn (HasAiChat.php:461) so a mid-stream toggle cannot swap providers.
- **Dual tool catalogue**: `AiToolDefinitions.php:59–63` emits Anthropic shape, `XaiToolDefinitions.php:188` emits OpenAI function objects. Neither holds schemas in code; both read `fyn-memory/procedural/tool_schema/`, the xAI side asking for the `.xai.md` variant (:140). Order constants (AiToolDefinitions.php:70–133; XaiToolDefinitions.php:66–122) are guarded byte-for-byte by golden-master tests, and a malformed corpus body is skipped with a `report()` rather than emptying the catalogue (AiToolDefinitions.php:171–189).

2.4 Tools

Assembled from `AiToolDefinitions.php:70–133`; executed by one dispatch match in `CoordinatingAgent.php:1166–1233`; classified read, write or handoff for the audit chain at :1578–1593.

- **Reads**: `navigate_to_page`, `list_records`, `list_goals`, `list_life_events`, `get_module_analysis`, `get_recommendations`, `search_conversation_index`, `get_tax_information`, `generate_financial_plan`, `get_subscription_status`, `list_invoices`, `get_current_plan`, plus one synthesised `fetch_{pointer_id}` per tool-mode pointer (:224–241).
- **Writes**: `create_what_if_scenario`, `create_goal`, `create_life_event`, `create_savings_account`, `create_investment_account`, `create_holding`, `create_pension`, `create_property`, `create_mortgage`, `create_protection_policy`, `create_asset`, `create_liability`, `create_estate_gift`, `create_will`, `update_will`, `create_power_of_attorney`, `update_power_of_attorney`, `create_family_member`, `create_trust`, `create_business_interest`, `create_chattel`, `update_record`, `delete_record`, `update_profile`, `set_expenditure`; eight campaign `capture_*` tools; five onboarding grouped-extract `capture_*` tools.
- **Handoff, never user-visible**: `delegate_to_capture`, `capture_complete` (HandoffContract.php:22–24).
- Preview mode strips every write group (AiToolDefinitions.php:29–40).

Write safety: three enforcement points, one list. `AdviceFyn::WRITE_TOOLS` (AdviceFyn.php:170–203) is the single denylist. (1) Catalogue strip by `array_diff` (:850), so the model never sees a write tool in advice mode. (2) Dispatch rejection through the typed-action allowlist (Actions/SurfaceAllowlist.php:47–54, called at HasAiChat.php:940–945); a denied call never executes, writes an audit row with `status: stripped` and returns a safe observation (:1697–1721). (3) `Ground/GroundGate.php:44–51`, the standalone predicate, parity-tested against the same list. `create_what_if_scenario` is on the list because it persists a row (:188–193); `navigate_to_page` is on it because the model was using it as an escape hatch for write intents and fabricating success (:194–202). The capture side re-adds every write tool except `navigate_to_page` (OnboardingChatDirector::captureToolSet(), :8472–8481) so the strip never creates a dead end.

How a write happens from advice mode:

```
LLM emits delegate_to_capture (reason, entity_types)
-> CoordinatingAgent yields {type: handoff} (CoordinatingAgent.php:1172-1176)
-> HasAiChat turns it into a synthetic SSE frame (HasAiChat.php:993-1000)
-> FynLoop::interceptHandoff consumes it, never forwarded (FynLoop.php:629-734, INV-2.4.1)
-> OnboardingChatDirector::handleInlineCapture
-> the same direct-write handlers in CoordinatingAgent
```

Supporting contracts: [KycGateChecker.php:32-69](#) (checks only what the primary classification needs); [RecordDuplicateChecker.php:44-91](#) (suppresses capture only when every extracted entity already exists); [DuplicateAcknowledgement.php:53-63](#) (a deterministic, non-model acknowledgement, because model phrasing had said "I've added these" when nothing was added); [HandoffPayloadValidator.php:16-31](#) (missing `entity_types` terminates the turn with a `handoff_error` frame); [ToolResultContract.php:45-143](#) (per-module required keys, drift throws rather than truncating); [StructuredResponseValidator.php:72-175](#) (flags acronyms, record IDs, emoji and glyphs, jargon, filler openers, advice with no £ amount, HTML, leaked context; flags but does not block); [Fyn/RecaptureGuard.php:243](#) (the one place that decides what happens when Fyn captures a record the user already has, extracted after twenty-four handlers were found failing differently).

2.5 Dispatch, endpoint, stream and loop

One endpoint for every surface. [routes/api.php:1507-1530](#) defines the `ai-chat` group behind `auth:sanctum` and `throttle:60,1`: list, create, show, delete conversations; `POST /conversations/{id}/messages (:1514, idempotent)`; stream and cancel a queued message; resumption; `POST /conversations/{id}/action`; onboarding status and start. iOS posts to exactly these paths ([ios-native/Fynla/Features/Fyn/FynClient.swift:84-223](#)); web and `/m` the same. The only surface signal on the wire is the `X-Fynla-Forms: 1` header ([AiChatController.php:1074-1078](#)), which native does not send yet, so native keeps the typed prompt for a form turn ([OnboardingChatDirector.php:135-141](#)).

The dispatch predicate has one home, [ContextualConversation/ConversationModeResolver.php:12-35](#), called from all three stream entry points ([AiChatController.php:277, 456, 947](#)). It routes to the writing director only when the Fyn flow is enabled, the conversation is not a surface action, and either the conversation is an onboarding one with a non-null step, or the user is not onboarding-complete or has an active campaign and has a non-null step. A completed or paused user routes to read-only advice even inside an onboarding conversation; the comment at [:26-29](#) records the live incident (csjones, conversation 245, 2026-09-19) that fixed this.

SSE contract. Every frame goes through one writer, [AiChatController::writeClientEvent\(\) \(:1060-1072\)](#), as `data: {json}\n\n`. Frame types: `content`, `done`, `thinking`, `title`, `tool_use`, `navigation`, `quick_replies`, `skip_link`, `fill_form`, `form_received`, `capture_form`, `capture_form_errors`, `capture_write_result`, `capture_complete`, `entity_created`, `entity_updated`, `entity_deleted`, `onboarding_advance`, `onboarding_field_captured`, `onboarding_layout_change`, `onboarding_complete`, `onboarding_capture_error`, `conversation_created`, `resume`, `level_up`, `token_limit`, `consent_required`, `handoff_error`, `error`, `action`. Never sent: `handoff` (consumed internally, [FynLoop.php:737-754](#)) and any `persona_state_change` (none exists). The four record-card frames are withheld for the whole of onboarding, in one place, for every surface ([AiChatController.php:41, 1062-1064](#)).

The agentic loop ([FynLoop::run\(\) \(:194-299\)](#)): build the planner prompt with the procedural corpus, recalled episodes and rubric ([:319-333](#)); emit `thinking`; loop up to the cycle cap calling [Planner::plan\(\) \(:2454-2505\)](#) and dispatching the typed action: `no_action` defers, `learn` stages and re-plans, `retrieve` re-plans under a budget of 3, `reason` or `ground` runs the streamed reasoner and returns. `CYCLE_CAP = 8 (:67, per-mode override in config/fyn.php:37-41)`. A planner failure degrades to `reason` rather than erroring the turn ([Planner.php:88-95](#)). The reasoner's own tool-use loop is [HasAiChat.php:385](#) onward: token-budget check, classification and KYC, prompt build, model selection and soft-degrade, tool-list resolution, a tool-call cap, then the provider loop with the typed-action gate at [:940-953](#).

Concurrency and consent. A 300-second per-conversation lock serialises turns ([AiChatController.php:241](#)); turns behind an in-flight one are queued to a depth of 3 with a 10-minute TTL ([config/fyn.php:27-30](#); [Loop/ConcurrentTurnQueue.php:29-44](#)). A fatal mid-stream error flags a resumption offered next session ([Loop/ResumptionService.php:23-60](#)). AI consent is gated at entry on every stream endpoint and re-checked at most every two seconds in-stream ([:306-315](#)).

2.6 How Fyn integrates with the module agents

- Engine-call level.** [AdviceFyn.php:101-126](#) maps every query classification to `holistic`, `module` or `factual`; unmapped types default to `factual` so production can never regress to the most expensive path ([:142-149](#)).
- get_module_analysis** -> [CoordinatingAgent::handleModuleAnalysis\(\) \(:2454-2505\)](#) picks one of the seven module agents (or `orchestrateAnalysis` for holistic), emits an `EngineCalled` eval event and summarises the raw agent output for the model ([mappedModuleAnalysis\(\) \(:534\)](#)).
- get_recommendations** -> [handleRecommendations\(\) \(:2737-2772\)](#) returns the ranked recommendations from `orchestrateAnalysis`, the composed tax plan from [ComposedTaxPlanService](#), and records strategy-id provenance and the plan digest into the request-scoped collector. The pointer path ([Pointers/Handlers/RecommendationHandler.php:24-44](#)) is pinned byte-equal to the same digest so tool and pointer cannot disagree.
- Plan pointers** ([CrossModulePlanHandler.php:21](#), [ModulePlanHandler.php:22](#) and five subclasses, [TaxAllowanceHandler.php:17](#), [UserFinancialHandler.php:13](#)) prefetch a module's composed plan, the allowances or the records summary into `<live_data>` before the model sees the turn.
- Context assembler** ([FynContextAssembler::build, \(:64-334\)](#)): always tax year, name, situation line, profile, current page, surface action, known facts, procedures, remembered episodes, knowledge, live data, overlays and FCA blocks; POSITION bucket adds `<financial_context>` and `<existing_records>` ([:219-224](#)); READINESS adds the per-user READY/BLOCKED module matrix ([:226-236](#)); CAPTURE adds the capture or verify-edit instructions ([:305-318](#)). The assembled block replaces the last user message in memory only; the stored row keeps the raw message ([HasAiChat.php:1632-1681](#)).

2.7 Cost, audit and eval

- **Cost:** `Cost/AiCostCalculator.php:27–49` prices input, output, cache-read and cache-write from `config/ai_pricing.php` in GBP; an unpriced model returns `priced: false` rather than £0. `AiCostAttributionService.php:27–38` writes one `ai_cost_attributions` row per model call; `FynLoop` records planner (`:504–544`) and reasoner (`:555–596`) stages separately, each carrying session mode, action, cycle, procedural version and cost.
- **Budgets:** `HasAiGuardrails.php:96–139`: a rolling seven-day total against the tier's weekly budget soft-degrades the model and prepends a notice (`HasAiChat.php:459–463`); a daily hard backstop surfaces a `token_limit` frame (`:404–414`). Both read from `TierConfigurationStore`; preview personas are never metered.
- **Audit chain:** `AuditChainService::append()` (`:89`) hashes fixed fields with the previous row's hash, HMAC-signs, inside a locked transaction (`:11–18`); deep-ksorts JSON because MySQL reorders keys (`:28–36`); throws if `app.ai_audit_hmac_key` is missing (`:52–69`). Written at dispatch (`CoordinatingAgent.php:984–991`), completion (`:1545`), rejection (`HasAiChat.php:1699–1706`) and per turn as an `__episode__` attestation (`:1525, 1575`). Verified weekly by `ai:audit:verify-chain`.
- **Eval:** `EngineCalled` and `GateChecked` events feed the HTTP-driven eval harness (`app/Services/Eval/`), with recall and precision floors of 95 per module (`config/fyn_eval.php:27–40`). No eval sessions are recorded locally or on production today (`eval_recording_sessions` is empty in both).

2.8 Where Fyn renders

Surface	Files
Web	<code>resources/js/components/Shared/AiChatPanel.vue</code> , <code>AiChatButton.vue</code> , <code>components/Fyn/FynOnboardingChat.vue</code> , <code>FynQuickReplies.vue</code> , <code>store/modules/aiChat.js</code> , <code>aiFormFill.js</code> , <code>services/aiChatService.js</code> ; entry points <code>layouts/AppLayout.vue</code> , <code>components/SideMenu.vue</code> , <code>views/Dashboard.vue</code> , <code>views/GamifiedDashboard.vue</code>
/m	<code>resources/mobile/mixins/onboardingChat.js</code> (the single SSE consumer, <code>:375–590</code>), <code>components/FynCaptureForm.vue</code> , <code>MobileChrome.vue</code> , <code>views/ConversationHistory.vue</code> , <code>views/Dashboard.vue</code> , <code>utils/fynText.js</code> , <code>utils/captureFormState.js</code> , <code>api.js</code>
iOS	<code>ios-native/FynIa/Features/Fyn/FynView.swift</code> , <code>FynClient.swift</code> , <code>FynEvent.swift</code> (single consumer, <code>:55–133</code>), <code>FynEventReducer.swift</code> , <code>FynConversationModel.swift</code> , <code>FynMessageView.swift</code> , <code>FynComposerView.swift</code> , <code>FynQuickRepliesView.swift</code> , <code>FynCaptureConfirmationView.swift</code> , <code>FynModels.swift</code> , <code>ConversationHistoryView.swift</code> , <code>ConversationHistoryModel.swift</code> , <code>Core/FynEditing/FynEditIntent.swift</code>

3. Memory: how it is designed

Fyn implements the CoALA model (Cognitive Architectures for Language Agents). The design statement is `fyn-memory/README.md:1–17`; paths are configured once in `config/fyn.php:49–59`. The governing sentence, at `README.md:17`: "**Memory holds pointers, not copies.**" Anything with a live owner (a tax allowance in `TaxConfigService`, a balance in an account row, a recommendation from the engine) is never frozen in memory; memory holds a pointer that says which source owns it and how to fetch it at the moment of need.

3.1 The four stores

Store	What it holds	Writer
Working	The turn: user, message, route, session mode, classification, KYC result (<code>FynTurnContext.php:15</code>), mapped onto four context buckets IDENTITY, POSITION, READINESS, CAPTURE (<code>ContextBucket.php:15</code> ; <code>FynContextSelector.php:19–35</code>). Onboarding gets IDENTITY + CAPTURE; a factual advice turn gets IDENTITY only; everything else gets IDENTITY + POSITION + READINESS.	<code>FynContextAssembler::build</code> (<code>app/Services/AI/Fyn/FynContextAssembler.php:47,66</code>)
Procedural	<i>How Fyn works</i> : the pointer registry (9 pointers, <code>fyn-memory/procedural/pointers/</code>), one root procedure <code>recommendation-routing.md</code> , two system-prompt overlays (<code>system_prompt_overlay/general/</code>), one onboarding workflow (<code>workflow/onboarding/fyn-onboarding.v1.md</code>), and roughly 50 tool schemas each with a <code>.md</code> and <code>.xai.md</code> variant (<code>tool_schema/</code>)	CSJ, by commit. Never agent-generated (<code>README.md:59–61</code>)
Episodic	Two separate stores. (a) CoALA episodes: per-user markdown written when the planner emits <code>learn</code> with <code>store=episodic</code> (<code>FynLoop.php:237,341</code>), governed by <code>episodic/RUBRIC.md</code> . (b) Forensic episodes: one <code>.md</code> blob per assistant message with the verbatim system prompt, assembled context, tool calls and results (<code>EpisodeBlobData.php:10,35</code>), plus five columns on <code>ai_messages</code> (<code>procedural_version</code> , <code>semantic_snapshot_id</code> , <code>fetch_provenance</code> , <code>blob_md_path</code> , <code>blob_md_sha256</code> ; migration <code>2026_06_01_000001</code>)	(a) Fyn via <code>FynMemoryStore::writeEpisode</code> (<code>:209</code>); (b) <code>HasAiChat::persistEpisode</code> (<code>app/Traits/HasAiChat.php:1525</code>), which also appends a signed <code>__episode__</code> attestation to the audit chain (<code>:1576–1584</code>)
Semantic	Knowledge with no live owner . Global: <code>fyn-memory/semantic/{allowance,fca,house_view,product,tax}/</code> ;	Global: CSJ by commit. Per-user: only <code>SemanticFactPromoter::approve</code> from a human-approved

Store	What it holds	Writer
	today only <code>house_view/</code> has content (20 strategy-stance files), the other four hold <code>.gitkeep</code> . Per-user: runtime facts about one person under <code>storage/app/memory/semantic-user/{userId}/</code>	proposal (<code>app/Services/AI/Learning/SemanticFactPromoter.php:18</code>)

3.2 The pointer model

A pointer is a markdown file of routing only, with no values and no fetch code (`fyn-memory/procedural/pointers/README.md`). `PointerRegistry.php:17` loads them fail-closed: an unknown mode, a prefetch pointer without triggers, or a pointer naming an unregistered handler all throw (`:112,117,122`), so the code handler must ship before the markdown. Three modes (`Pointer.php:22-30`):

- **prefetch**: a lowercase substring match of each trigger against the user's message (`PointerRegistry.php:60-78`) runs the handler and injects the result into `<live_data>` before the model sees the turn (`FynContextAssembler.php:151-165`).
- **tool**: exposed to the model as a callable tool (`XaiToolDefinitions.php:200`; executed through `CoordinatingAgent.php:1647-1659`).
- **both**.

`FetchHandlerRegistry.php:10` is the closed whitelist of ten code handlers under `Pointers/Handlers/`. `FetchDispatcher.php:16` runs one, records provenance (`:33`) and on any throwable degrades to null with a `report()` rather than breaking the turn (`:27-31`). Example: `TaxAllowanceHandler.php:30` reads the ISA and pension allowances live from `TaxConfigService` and the user's subscriptions from `ISATracker`, stamping the active tax year as the source version (`:57`). This is `CLAUDE.md` Rule 2 (no hardcoded tax values) expressed as memory architecture.

Nine pointers exist: `isa-annual-allowance`, `recommendations`, `user-financial-position`, and six plan pointers (cross-module, estate, investment, protection, retirement, savings).

3.3 Retrieval is sparse, by decision

There is no embeddings table and no vector column anywhere. `RecallScorer.php:12` records the decision: dense embeddings deferred until roughly 500 concurrent users (CSJ, 2026-06-01). `SemanticRetriever::retrieve` drops stopwords (`:31-41`), filters by effective date before ranking (`:68`), counts word-boundary token matches with plural variants (`:181,194`), and admits a fact only when it matches at least two distinct query tokens (`:61,83`). The docblock at `:18-20` records why: a substring scorer had been loading four full strategy bodies on every English turn because "the" scored 50 to 67 per file. `FynMemoryStore::matchingProcedures` (`:86`) and `SparseRecallScorer` (`:18`) use the same grammar and stopword list.

3.4 "Known facts" about the user

`MemoryRetrieverService` (`app/Services/AI/MemoryRetrieverService.php:13-48`) is not a CoALA store; it assembles the `<known_facts>` block from four layers with add-only merging (`mergeNewKeys`, `:334`), so a lower layer can never overwrite an authoritative one:

1. Authoritative database fields: name, date of birth, marital and employment status, employer, occupation, dependants, record counts (`:86`).
2. Parked onboarding facts from `ai_conversations.onboarding_parked_facts` (`:151`).
3. The current conversation, re-extracted from the last six user messages (`:215`).
4. The conversation index: `prior_topics` and `prior_intents` from the five most recent other summarised conversations, capped at five, deliberately hint-shaped (`:254`).

The block closes with "Do not ask the user for any field above." (`:306-325`). Callers: `AdvicePromptBuilder.php:65`, `FynContextAssembler.php:51`, `OnboardingPromptBuilder.php:38`, `OnboardingChatDirector.php:120`.

3.5 Conversation persistence and summaries

- `ai_conversations` (migration `2026_02_27_200001`) with later columns `persona_state`, `onboarding_parked_facts`, `summary`, `topics`, `entities_mentioned`, `intents_stated`, `summarised_at`, `pending_resumption`. `ai_messages` (`2026_02_27_200002`) with `system_prompt`, `assembled_context`, `persona`, a `status` enum (queued, processing, answered, cancelled, expired) and the five episode columns. Model classes `app/Models/AiConversation.php:15` (soft deletes) and `AiMessage.php:12`.
- Conversations are created `active` (`AiChatController.php:80`), reactivated on use (`:223`), paused by a three-minute scheduled sweep (`Kernel.php:34-36`) and reopened when the user next sends. `ResumptionService.php:17` flags a conversation that ended unfinished and offers it next session (`:23,54`; routes `routes/api.php:1523-1524`).
- **History filter**. `HasAiChat::buildMessageHistory` (`:1786`) sends the model the last 20 user and assistant messages (`MAX_HISTORY_MESSAGES`, `:98`) and drops assistant rows flagged `is_retry` or containing the canned refusal (`:1807-1813`). The comment at `:1799-1806` records the live incident that motivated it: once dead-end rows were in the history, the model pattern-matched and refused a full entity sentence (production conversation 843, 2026-09-11). Those rows stay visible to the user; only the model-facing history is filtered, in this one place.
- **Summaries**. `ConversationSummariser.php:43` compresses up to 50 messages into the four index columns (`:85-91`) using xAI directly (`:45,137`), temperature 0, JSON output, forbidden from inventing figures (`:150-154`). Dispatched at end of onboarding (`OnboardingChatDirector.php:7674`) and by `ai_conversations:summarise-stale` (`SummariseStaleConversationsCommand.php:34`), which skips in-flight onboarding conversations (`:53-65`). Summaries are re-used by known-facts layer 4 and by the `search_conversation_index` tool (`CoordinatingAgent.php:2657-2684`).

3.6 Learning

Off by default. `config/fyn.php:68` sets `learning_enabled` false and the variable is in neither `.env` nor `.env.example`. When on, three write paths open and none auto-applies: `ProposedFactSynthesiser.php:16` (from the summariser, forbidden from extracting money amounts or anything with a live source, `:28-36`), `FynLoop::stageProposedFact` (`:360`) and `FynLoop::stageProcedureAmendment` (`:388`). All land as `pending` in `proposed_semantic_facts` / `proposed_procedure_amendments`; promotion is `fyn:semantic:promote {fact} --reviewer=`, human-gated, and never touches the global corpus (`SemanticFactPromoter.php:10-12`). The planner schema warns the model that `learn` is not for recording the user's financial data (`Planner.php:311,315`; `FynLoop.php:82`).

3.7 Erasure

`fyn:user:erase` (`FynUserErase.php:27`) deletes blobs, then `ai_messages` and `ai_conversations` in a transaction, then per-user semantic facts; dry-run unless `--force`. `RetentionPurgeService.php:47-53` deletes the CoALA episode tree and the blobs. See §6 for the gap between them.

3.8 Caches around memory

`ai_existing_records_{user}` at 60 seconds and `ai_financial_context_{user}_{classification}` at 120 seconds (`AdvicePromptBuilder.php:763,476`), cleared by `AdvicePromptCacheInvalidator::forUser (:35-65)` after any write so a user who has just added an ISA is not told they have none. Procedural corpus cache keyed by directory signature (`ProceduralCorpusLoader.php:30-32,67-72`). `PointerRegistry` is a singleton (`AppServiceProvider.php:141`); `FynMemoryStore` is deliberately not, so its memo cannot outlive a turn (`:36`).

3.9 One memory for every surface

Web (`resources/js/services/aiChatService.js`), /m (`resources/mobile/mixins/onboardingChat.js:93-723`) and native iOS (`ios-native/FynIa/Features/Fyn/FynClient.swift:84-223`) call the same `api/ai-chat/*` routes (`routes/api.php:1507-1529`). One conversation store, one history, one memory of every type, one pointer registry. No per-surface partition exists. This is the architectural fact behind `CLAUDE.md` Rule 20.

3.10 One turn, end to end

Before the model call: the conversation is set active; `FynTurnContext` and the bucket selector describe the turn; the planner (`FynLoop::plannerSystemPrompt, :315`) sees the full procedural corpus, the user's ranked episodes and the rubric; the planner runs up to the cycle cap of 8 (`config/fyn.php:37-41`); `FynContextAssembler::build (:66)` lays down profile, `<known_facts>`, `<procedures>`, `<remembered>` (five most recent episodes), `<knowledge>`, `<live_data>` (prefetch pointers), overlays, FCA blocks and the bucket-gated financial, records, readiness, KYC and billing layers; the last 20 filtered turns are attached; tool schemas come from the procedural corpus.

After: the assistant message is persisted with its prompt and context; `persistEpisode` writes the blob, the five columns and the audit attestation; a `learn` may append a CoALA episode; with learning on, a proposal may be staged; any write tool clears the prompt caches; later the scheduler pauses and summarises the conversation, feeding the next one.

4. Workforce agents (`.claude/agents/`)

Twenty agent definitions. They share one constitution (`workforce/core/constitution/`, eight files), one charter (`workforce/core/charter.md`), one registry (`workforce/core/registry/`), one board (`workforce/ops/board/`, 347 items), one event log (`workforce/ops/log/*.jsonl`) and one command (`workforce/ops/wf.sh`: log, claim, move, handoff, brief, status). Precedence: `CLAUDE.md` first, then the perimeter file, the design guide, product strategy (advisory), the charter (`workforce/core/index.md`).

4.1 Governance and leads

Agent	Personality and role	What it does	Integrates with	Evidence
chief-of-staff ("Myrtle")	Named colleague, signs every external message as Myrtle. Judges, never builds. "A named gap is a pass. A hidden gap is a failure."	Mission intake (at most five questions), judgement on four axes (goal fit, trunk fit, quality bar, blast radius), gates, liveness probes at 45 minutes of silence, the 17:30 daily brief. Sole reader and voice in Slack, WhatsApp, GitHub issues, email; classifies everything overheard as noise, information, issue, request, question or trunk conflict.	Reads <code>.goal</code> , the board and the trunk; assigns leads; judges Quality's evidence pack; escalates to founders. Never approves its own gates.	<code>.claude/agents/chief-of-staff.md</code> ; <code>charter.md</code> §13; log; 56 events, 23 briefs
build-lead	The deliverer. "Loop until correct"; never self-certifies.	Claims board items, runs the six-source prior-art check (three outcomes: none, route, extend), reads vault docs before module work, names surfaces (web, /m, iOS) on every item, builds, writes a handoff note, hands to Quality.	Dispatches <code>frontend-developer</code> , <code>database-optimizer</code> , <code>premium-ui-designer</code> , <code>excalidraw</code> ; hands to <code>quality-lead</code> .	<code>.claude/agents/build-lead.md</code> ; log; 126 events; owns 244 board items
quality-lead	The merge gate. "No pack, no merge." Never verifies its own work.	Runs the code (Pest, Pint, reviewers, lint hooks) and drives every journey as a user in Playwright, filling and submitting every form, reading state back from the database, capturing console and timestamped screenshots. Authors the evidence pack at <code>workforce/branches/<type>.<slug>/evidence/</code> . Weekly <code>tech-debt-full</code> .	Receives from build-lead; pack judged by chief-of-staff; routes iOS to CSJ's device (Playwright cannot drive SwiftUI).	<code>.claude/agents/quality-lead.md</code> ; <code>08-process.md</code> §2; <code>handoffs/quality-lead/</code> (4 certifications)

Agent	Personality and role	What it does	Integrates with	Evidence
compliance-lead	Applies written rules; never determines what the law requires. Two outputs only: "no issues found within my competence" or "flagged, with reason and dated source". Never says "compliant".	Hard-blocks diffs to tax services, AI prompt files and public claims; screens all outbound content against the seven regulatory rules in <code>ComplianceRules.php</code> ; maintains the dated source register; drafts precise questions for a lawyer (<code>05-perimeter.md</code> §6).	Block overridable only by a founder, not the chief-of-staff.	<code>.claude/agents/compliance-lead.md</code> ; log: 16 events
design-lead	Guardian of the palette, the no-scores rule and the icons rule. Escalates to Azlan, not CSJ.	Enforces Rules 8, 11, 12, 15 on every UI diff; monthly design audit; copy in the Functional register. Never strips the approved <code>/m</code> gamification layer.	Dispatches <code>premium-ui-designer</code> , <code>ui-graph</code> , <code>ux-writing-expert</code> , <code>excalidraw</code> .	<code>.claude/agents/design-lead.md</code> ; owns 8 board items
growth-lead	"You may draft anything. You may publish nothing." Persuasive register, but constants C1 to C7 never bend.	Marketing content, campaigns, landing pages, SEO, email; routes articles into the existing Drive -> <code>InsightArticle</code> pipeline with its approve-to-production button rather than building a second approval path. Targets personas, never income or age bands.	Escalates tone to Azlan, claims and pricing to Brett; screened by compliance-lead.	<code>.claude/agents/growth-lead.md</code> ; no log events yet
intelligence-lead	The measurer. Every figure with source, method, confidence and what it does not show.	North star Paid Active Households (one paid subscription, logged in within 30 days, three or more modules populated); guardrails (churn, CAC payback, net revenue retention, support load, Fyn AI cost under 12% of ARR, crash-free 99.5%); daily tick, weekly narrative. Plausible for traffic only, the database for anything per-user.	Breach goes to chief-of-staff immediately.	<code>.claude/agents/intelligence-lead.md</code> ; no log events yet
product-lead	Applies one test to everything: which persona is this for, and what does it do for them. Escalates to CSJ.	Specs, PRDs (<code>prd-writer</code> , which needs a spec and a plan first), roadmap, prioritisation; acceptance and browser checkpoints written before code; surfaces named individually.	Dispatches <code>plan-and-build</code> , <code>product-manager</code> ; prior-art check via <code>capabilities.md</code> .	<code>.claude/agents/product-lead.md</code> ; owns 5 board items

4.2 Infrastructure roles

Agent	Personality and role	What it does	Evidence
cartographer	"A map nobody owns goes stale, and a stale map is worse than none." Reads the enforcing layer (a resolver), not the descriptive one (a seeder).	Owns <code>workforce/core/registry/capabilities.md</code> in three dimensions: capability, surface, consumers. Serves role-scoped views (full in-domain, one line adjacent, everything queryable). Records absence as well as presence (crypto is not modelled). Freshness on merge, on discovery, quarterly.	<code>.claude/agents/cartographer.md</code> ; <code>capabilities.md</code> §7 survey of 2026-08-13
quartermaster	Audits whether an agent was	Seven gap classes (access, tool, credential, context, knowledge, capability, contradiction); self-repair of silent, hung or thrashing	<code>.claude/agents/quartermaster.md</code> ; no log events yet

Agent	Personality and role	What it does	Evidence
	equipped, never whether its work was good. "You may make the machinery report more. You may never make it report less."	agents; doctrine interviews capped at five questions, each with a drafted proposal; weekly roster sweep.	
archivist	Keeps the knowledge tree honest. "A rule may only be created or changed in the trunk."	Nightly sweep: orphan, contradiction, staleness (runnable as <code>workforce/ops/sweep.sh</code>); exactly two outcomes for a contradiction (fix the branch, or amend the trunk through a founder); continuous fact-checking of counts and paths; quarterly doctrine review that prunes before adding; vault mirroring via <code>vault-sync</code> ; handovers; meeting ingestion.	<code>.claude/agents/archivist.md</code> ; log: 1 event; owns 3 board items
persona-tester	Enters a real household and proves the app tells it the truth, from both sides of the marriage. The persona file is the contract and is never edited to make a test pass.	Three passes per persona (web forms; Fyn on <code>/m</code> ; Fyn on native iOS), each entering, verifying on web, <code>/m</code> and iOS for both the primary and the spouse account, then tearing down. Owns the defect lifecycle: document, route, fix, re-test, PR to dev, re-test on dev, sign off.	<code>.claude/agents/persona-tester.md</code> ; <code>tests/Persona/20-08-2026_run/reports/</code> (12 reports), <code>07-09-2026_new_user_run/reports/</code> (5)

4.3 Specialist tools

Agent	What it does	Evidence
security-reviewer	Audits auth flows, endpoints, form handling and financial-data exposure; required in the evidence pack for any diff touching auth, financial data or user input.	<code>.claude/agents/security-reviewer.md</code> ; <code>08-process.md</code> §2.1
tax-compliance-reviewer	Verifies tax code against HMRC rules through <code>TaxConfigService</code> , flags hardcoded values; required on tax services or projections. Its frontmatter says active year 2025/26 while <code>CLAUDE.md</code> says 2026/27 (see §6).	<code>.claude/agents/tax-compliance-reviewer.md</code> ; <code>handoffs/W-0008-W-0205/tax-compliance-reviewer-2026-08-25.md</code>
database-optimizer	Slow queries, indexes, N+1, schema scaling.	<code>.claude/agents/database-optimizer.md</code>
premium-ui-designer	Polish within <code>fynlaDesignGuide.md</code> : spacing, motion, states, depth; never new colours, gradients or icons.	<code>.claude/agents/premium-ui-designer.md</code>
ux-writing-expert	Microcopy: errors, empty states, labels, onboarding copy. Its frontmatter colour is <code>orange</code> , a token Rule 8 bans in product UI (see §6).	<code>.claude/agents/ux-writing-expert.md</code>
frontend-developer	Converts public Vue pages to standalone HTML through the <code>html-template</code> skill.	<code>.claude/agents/frontend-developer.md</code>
laravel-stack-deployer	Deploy runbook executor; defers to <code>CLAUDE.md</code> and SiteGround specifics.	<code>.claude/agents/laravel-stack-deployer.md</code>
product-manager	Personas, user stories, backlogs; documentation only, never code.	<code>.claude/agents/product-manager.md</code>

4.4 What they share

- **The trunk.** Constitution (precedence, mission, values, hard nos, voice, perimeter, commercials, quality bar, process), charter, registries (systems, storage, comms, tools, access, people, rhythm, meetings, capabilities, sources). Rules are made only in the trunk; branches apply them (`archivist.md`).
- **The board and formats.** `workforce/ops/FORMATS.md` fixes the work-item frontmatter (`status`, `surfaces`, `prior_art_checked`, `handoff_to`, `constitution_refs`). An item cannot be claimed without a prior-art check; git rejects a second claimer; an item cannot move to handoff with an empty note.
- **The event log.** `wf.sh` appends JSON lines; the brief, liveness monitoring and mission control read it. Today's counts: build-lead 126, chief-of-staff 56, compliance-lead 16, quality-lead 3, archivist 1.
- **Hooks.** `.claude/hooks/` enforce the standing blocks mechanically: dangerous-command guard, env guard, prod guard, tax-hardcode check, design lint, `/m` parity check, oversight guard (protects the hooks, settings, agent definitions, constitution and `CLAUDE.md` from being narrowed), workforce guard.
- **Skills.** `vault-context` before module work, `fyn-architecture` before Fyn work, `verify-m`, `release`, `app-map`, `excalidraw`, `prd-writer`, `tech-debt-session`.

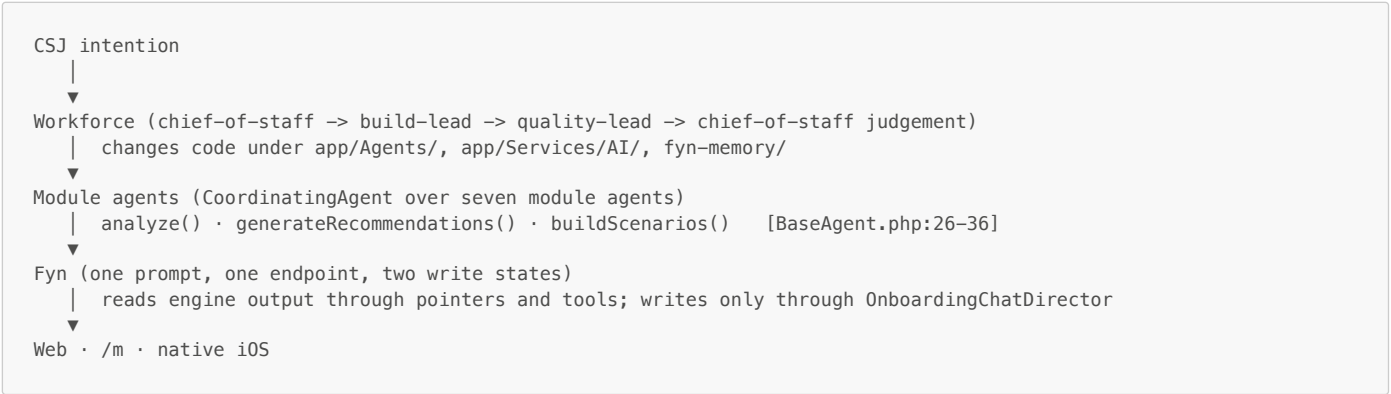
4.5 How the workforce remembers

Memory	Where	Notes
Doctrine and registries	<code>workforce/core/</code>	Only a founder ratifies a change; the Archivist fact-checks counts and paths continuously
Work state	<code>workforce/ops/board/</code> , <code>missions/</code> , <code>gates/</code> , <code>handoffs/</code> , <code>gaps/</code> , <code>interviews/</code> , <code>queue/</code>	347 items; 3 missions; 4 gates; 66 handoff folders
Event history	<code>workforce/ops/log/YYYY-MM.jsonl</code>	Append-only
Evidence	<code>workforce/branches/<type>/<slug>/evidence/</code> , <code>.smoke-evidence/</code> , <code>tests/Persona/<run>/reports/</code>	Merge evidence packs and run reports
Session continuity	<code>handover/<Month>/<DD>/</code> , <code>.remember/</code> (<code>now.md</code> , <code>recent.md</code> , <code>archive.md</code>), <code>MEMORY.md</code> index in the Claude project directory	<code>session-start</code> reads what <code>session-end</code> and <code>context-handover</code> write; a handover "never advances status"
Readable mirror	<code>fynlaBrain/</code> Obsidian vault (1,514 documents)	A mirror, not a source: where vault and trunk disagree, the trunk wins (<code>storage.md</code> §2)

4.6 What is live and what is not

From the log and board this morning: build-lead, chief-of-staff (briefs, decisions, gates), compliance-lead, quality-lead and persona-tester have produced work. Design-lead and product-lead are used directly without log events. Cartographer, archivist and quartermaster are built and lightly used; the nightly sweep is a script, not a scheduled process. Intelligence-lead and growth-lead have no log events and no board items. Myrtle's Slack channel has never connected: 12 `slack_failed` events, the latest yesterday (`log/2026-09.jsonl`), and the connector is recorded as unauthorised (`charter.md` §13.3).

5. How the three populations integrate



The rule that binds all three: every Fyn behaviour has one home, and a change is made once for every surface and every path (`CLAUDE.md` Rule 20). The workforce's "one home per rule" and the memory layer's "pointers, not copies" are the same principle applied to doctrine and to data.

6. Adjacent findings (reported, not fixed)

#	Finding	Evidence
1	<code>ComplianceRules::get()</code> defaults its tax year parameter to the literal <code>'2026/27'</code> , a hardcoded year in a prompt file (Rule 2).	<code>app/Services/AI/Prompts/ComplianceRules.php:15</code>
2	The episodic rubric is <code>version: 0</code> , <code>status: draft</code> , so <code>FynMemoryStore::rubric()</code> returns an empty string and the planner never receives the rubric it is meant to apply.	<code>fyn-memory/episodic/RUBRIC.md</code> ; <code>FynMemoryStore.php:147-149</code>
3	All 414 files under <code>fyn-memory/episodic/episodes/</code> locally are test artefacts (<code>cycle-N-learn-*</code> , <code>salience: 0</code>). There is no real episodic content in the local store.	<code>ls fyn-memory/episodic/episodes</code>
4	The two GDPR erasure paths do not overlap: <code>fyn:user:erase</code> skips the CoALA markdown episode tree; <code>RetentionPurgeService</code> skips the per-user semantic facts.	<code>FynUserErase.php:61-76</code> ; <code>RetentionPurgeService.php:47-53</code>
5	<code>tax-compliance-reviewer.md</code> states the active tax year as 2025/26; <code>CLAUDE.md</code> says 2026/27.	<code>.claude/agents/tax-compliance-reviewer.md</code> "Core Rule"

#	Finding	Evidence
6	<code>ux-writing-expert.md</code> frontmatter colour is orange; <code>chief-of-staff.md</code> is blue; <code>quartermaster.md</code> and <code>database-optimizer.md</code> are yellow. Not user-facing, but not palette tokens either.	<code>.claude/agents/*.md</code> frontmatter
7	<code>CoreIdentity.php</code> duplicates the identity, security and scope blocks of <code>FynSystemPrompt.php</code> for the legacy prompt architecture; the <code>fyn-architecture</code> skill says <code>legacy</code> is an emergency rollback path only. Two homes for one prompt text while it remains.	<code>app/Services/AI/Prompts/CoreIdentity.php:23-60</code> ; <code>FynSystemPrompt.php:70-216</code>
8	<code>ConversationSummariser</code> and <code>ProposedFactSynthesiser</code> call xAI over direct HTTP with a hardcoded endpoint rather than through <code>XaiClient</code> , and use the <code>vision_model</code> config key for a text task.	<code>ConversationSummariser.php:45,137</code> ; <code>ProposedFactSynthesiser.php:18,42</code>
9	CoreIdentity.php renders its placeholders literally. <code>{\ \$personality}</code> and <code>{\ \$responseFormat}</code> sit inside an interpolating heredoc, so the legacy rollback prompt ships with the literal text <code>{\ \$personality}</code> instead of the tone block. Latent, since <code>FYN_PROMPT_ARCH</code> defaults to <code>unified</code> .	<code>CoreIdentity.php:52,56</code>
10	Four SSE consumers in the web store, not equivalent. <code>aiChat.js</code> switches on event type at <code>:680, :1097, :1355</code> and <code>:1644</code> ; the queued-turn consumer at <code>:1097</code> has no <code>quick_replies</code> case, so a queued turn ending in bubbles would render none. /m and iOS each have one consumer. The pattern the <code>fyn-architecture</code> skill's Rule 20 note was written against. Not browser-tested this run.	<code>resources/js/store/modules/aiChat.js:680,1097,1355,1644</code>
11	Two provider readers skip the versioned cache key (<code>ai_provider</code> rather than <code>ai_provider:v{N}</code>); they agree today because the admin toggle writes both.	<code>AdviceFyn.php:835</code> ; <code>AiToolDefinitions.php:54</code> ; canonical reader <code>HasAiGuardrails.php:64-79</code>
12	A cache-tags argument is silently discarded. <code>BaseAgent::remember</code> takes three parameters, but Estate, Protection and Retirement build a <code>\$cacheTags</code> array and pass it as a fourth; PHP drops it, so no tag-based invalidation exists.	<code>BaseAgent.php:45</code> ; <code>EstateAgent.php:113</code> ; <code>ProtectionAgent.php:68</code> ; <code>RetirementAgent.php:108,295,832</code>
13	CoordinatingAgent::invalidateModuleCache forgets keys nothing writes (<code>v1_savings_{id}</code> , <code>v1_coordinating_{id}_analysis</code> and so on) while the agents write <code>savings_analysis_{id}</code> or <code>v1_savingsagent_{id}_analysis</code> . Same class of mismatch as W-0381, masked because <code>CacheInvalidationService</code> uses the right names.	<code>CoordinatingAgent.php:4856-4878</code> ; <code>BaseAgent.php:58-63</code>
14	One hardcoded tax value in the coordinator: the ISA-allowance conflict branch falls back to the literal <code>20000</code> when config is unavailable, with a comment naming 2025/26.	<code>CoordinatingAgent.php:501-503</code>
15	Dead methods: <code>TaxOptimisationAgent::generateRecommendations</code> and <code>::buildScenarios</code> have no production callers; <code>CoordinatingAgent::buildScenarios</code> is a stub returning "not yet implemented"; <code>CoordinatingAgent::generateRecommendations</code> has no caller; <code>BaseAgent::invalidateCacheForUsers</code> is called only from its own unit test.	<code>TaxOptimisationAgent.php:53,80</code> ; <code>CoordinatingAgent.php:341,349-356</code> ; <code>tests/Unit/Agents/BaseAgentTest.php:151,163</code>
16	Duplicate: <code>ProtectionStrategySource</code> states it builds "gaps + profile exactly as <code>ProtectionAgent::analyze()</code> does". Two mechanisms for one protection analysis. I COULD NOT VERIFY behavioural equivalence beyond the docblock.	<code>app/Services/Coordination/PlanSources/ProtectionStrategySource.php:44,72</code>
17	<code>EstateAgent</code> is absent from <code>RecommendationsAggregatorService</code> , so estate	<code>RecommendationsAggregatorService.php:33-38</code>

#	Finding	Evidence
	recommendations do not reach the recommendations API, focus cards or /m next actions through that path. May be deliberate (Estate is teaser on the free tier); recorded, not judged.	

7. I COULD NOT VERIFY

- Whether storage/app/episodic/ or storage/app/memory/semantic-user/ hold data on production; I read only the local repository and ran aggregate counts on production tables, not the filesystem.
- The bodies of the fyn:episodic:* and fyn:procedural:validate commands beyond their signatures.
- The /m and native clients' own caching of Fyn state beyond confirming they call the shared endpoints.